

Dynamic Real-Time Image Processing Pipeline Using FreeRTOS

An Academic Project Report This document presents a detailed explanation of the design, logic, and implementation of a real-time image processing pipeline developed using FreeRTOS. The primary focus is on timing control, task prioritization, and dynamic scheduling under strict real-time constraints.

1. Introduction Real-time embedded systems must satisfy strict timing requirements in addition to functional correctness. In multimedia and image processing applications, execution time variability can easily violate real-time deadlines. This project addresses these challenges by designing a deterministic image processing pipeline that adapts dynamically to timing disturbances. The system is implemented using FreeRTOS and OpenCV, running on a host machine to simulate embedded behavior. Although executed on a PC, the scheduling logic and task design follow embedded real-time principles.

2. System Overview The pipeline consists of five main tasks: Capture, Grayscale, Blur, Edge Detection, and Invert. Each task performs a single image processing operation and communicates through shared image paths. The Capture task always executes first and acquires an image from the camera. The remaining four processing tasks operate on the output of the previous stage. Their execution order is controlled purely by task priorities, allowing the pipeline to be reconfigured dynamically without modifying task code.

3. FreeRTOS Task Model FreeRTOS is a preemptive real-time operating system that schedules tasks based on priority. Higher-priority tasks always preempt lower-priority ones when ready to run. This behavior is fundamental to the pipeline design. Each image processing stage is implemented as an independent FreeRTOS task. Task synchronization is achieved using task notifications, which provide a lightweight signaling mechanism suitable for real-time systems.

4. Capture Task The Capture task has the highest priority in the system. Its sole responsibility is to acquire an image from the camera, crop it to a fixed region of interest, and store it to disk. After completing the capture operation, the task notifies all processing tasks and then deletes itself. This ensures that capture overhead does not interfere with pipeline timing.

5. Image Processing Tasks Each processing task performs exactly one image transformation. This modular design simplifies timing analysis and allows individual stages to be optimized or replaced independently. Execution time is measured inside each task using the FreeRTOS tick counter. This information is later used to compute the total pipeline time.

6. Invert Task and Random Delay The Invert task intentionally includes a random delay to simulate unpredictable real-world disturbances such as I/O stalls or computational overload. The delay is generated early, before task creation, and ranges from 0 to 90 ms. By knowing the delay in advance, the system can adapt its scheduling strategy before execution begins.

7. Dynamic Priority Adaptation Logic To ensure the pipeline completes within 100 ms, a priority adaptation mechanism is introduced. The logic increases the priority of the Invert task when its delay exceeds specific thresholds. If the delay exceeds 25 ms, the Invert task is promoted by one priority level. If it exceeds 35 ms, it is promoted further, and for delays greater than 50 ms, the Invert task becomes the highest-priority processing task.

8. Blocking and Concurrency The key insight of the design is that delaying a task early is less harmful than delaying it late. When the Invert task executes first and enters a blocked state, other tasks can proceed concurrently. This approach minimizes idle CPU time and improves overall schedulability, even in the presence of long delays.

9. Timing Analysis The total pipeline execution time is computed as the sum of all task execution times plus the random delay. The system prints a warning if the total exceeds the 100 ms budget. Experimental results show that early execution of delayed tasks significantly reduces deadline violations compared to static scheduling.

10. Conclusion This project demonstrates how dynamic task prioritization and blocking-aware design can be used to enforce real-time constraints in image processing systems. The techniques presented are applicable to a wide range of embedded systems where execution time variability is unavoidable.